FastAPI

Learning Sources
FastAPI Official Docs

Notes
-Notes originally created in Jan 2025 & completed July 2026

00. Intro to FastAPI

-Source: https://fastapi.tiangolo.com/

-FastAPI - a modern, high-performance web framework for building Python APIs with type hints

-Features - very high performance, on par with NodeJS and Go, fast to code in, fewer bugs, intuitive, easy to learn, short, robust w/ interactive documentation, and complying with OpenAPI specifications/standards.

-FastAPI requires pydantic (data validation library) and Starlette (an ASGI, async server gateway interface, for building Py async web services)

Install

```
cd <project directory>
python3 -m venv venv
source venv/bin/activate
pip install "fastapi[standard]"Create
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def read_root():
    return {"Hello": "World"}

@app.get("/items/{item_id}")
async def read_item(item_id: int, q: str | None = None):
    return {"item_id": item_id, "q": q}
```

Run

```
$ fastapi dev main.py              # starts dev server

# Serving at: http://127.0.0.1:8000
# API docs: http://127.0.0.1:8000/docs
```

Check it Out
-open *http://127.0.0.1:8000/items/5?q=somequery*

-Response is *{"item_id":5,"q":"somequery"}*, *where item_id* set as part of call to url (*/5*) and request params set by *?q=somequery*

-Check out headers, also. Notice, server is Uvicorn (an ASGI web server that loads and serves the application). No permission policies set, yet, also.

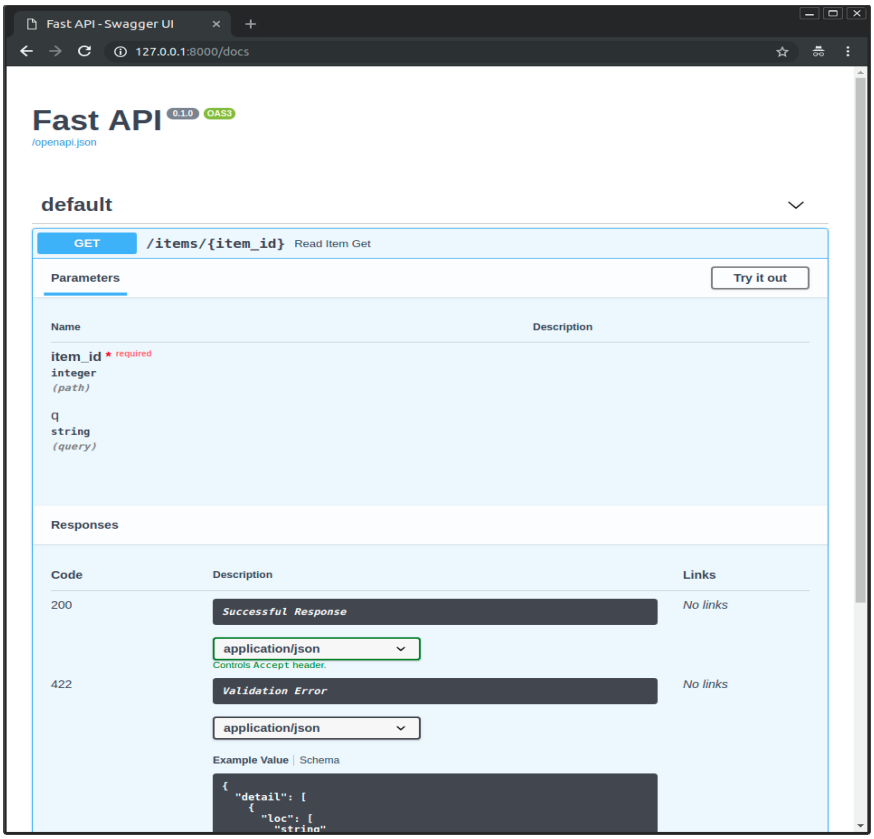
AI Agent Skill
-FastAPI has made a skill available for AI agents, including Claude Code, Codex, etc. to use. Install via:
pip install library-skills

What Just Happened
1. An app was created via creating an instance of FastAPI
2. Multiple HTTP GET paths were added, including one that allowed query params and resource IDs
3. FastAPI was called via querying path on the browser
4. FastAPI returned a JSON response

*Note: *path*, *endpoint*, and *route* all mean the same thing in API context and can be used interchangeably

Interactive Docs
-Go to: http://127.0.0.1:8000/docs

-FastAPI creates interactive Swagger UI docs for all paths. The docs show paths and allows you to make sample requests to them, with customized resource ids and params. Doc then shows request URL, response body, response headers, response code, etc.



-Alt UI docs also available via ReDoc at http://127.0.0.1:8000/redoc

Add More Functionality
-In *main.py, can add: from pydantic import BaseModel*

```
class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None

@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item):
    return {"item_name": item.name, "item_id": item_id}
```

-Dev server automatically updates, as soon as save code file, so can see new *PUT /items/{items_id}* path and space to enter in detailed request body to call path with:

```
{
  "name": "string",
  "price": 0,
  "is_offer": true
}
```

-Which, when called with valid test data (*{"name": "test_name", "price": 1234, ...}*) , responds with headers, 200 status code, and *{ "item_name": "est_name", "item_id": 1234 }* response body

Summary
-FastAPI lets you declare path request parameters / body as typed function parameters to an HTTP operation path declarations that wraps a standard python function.

-Simple request parameters can be defined using existing python types (ex. *item_id: int*) while complex params can be defined by python data classes (see *Item* example above)

-With each defined path, FastAPI provides:

- data validation - automatic and clear errors when data invalid. Validation even for deeply nested JSON objects.
- editor support - including auto-completion and type checks
- conversion of input data to python data/types, reading from JSON, path params, query params, cookies, headers, forms, and files
- conversion of output data from py data and types, to network data (as JSON), including conversion of py types, *datetime* objects, *UUID* objects, database models, and other types
- Automatically generated interactive docs with two user interfaces (Swagger UI and ReDoc)

00. Python Types

Type Hints & Benefits
-Python allows optional "type hints" to declare a type of a variable. Declaring types not only prevents bugs but creates clearer, more fixed models, and gives you IDE hints for code, after var is typed.

-Example of IDE benefits of type hinting: Function takes in untyped param *first_name*. You want to make sure it is upper case before printing it. Is the py function called *upper()* or perhaps *capitalize()?* You enter a '.' after the *first_name* variable, to see if the IDE will list you functions, but it doesn't. If you make *first_name* typed, with *first_name: str*, though, now the IDE sees the var's String methods. Boom, you find the method is called *capitalize()* when you hit '.' after the var, this time.

-Type hinting in function param syntax ex.: *def my_function(first_name: str, age: int)*

-More important than IDE auto-complete benefits is improved data validation, for wrongParamType, etc. otherwise potentially overlooked mistakes.

Py Types

-Need to know Py types: *str, int, float, list, tuple, range, dict, defaultdict, set, frozenset, bool, None*

-If need be, can cast from one type to another. Ex. *str(age)* where *age* is *int*

-Container types (*set, list, tuple, dict*) can also declare the types of items they hold, and can be nested

-Ex. of container w/ typed *items: list[str]*

-"Type parameters" are passed into container square brackets to determine contents' type.

-With tuple, since fixed length, can declare multiple types, each element its own type. Ex. *mixed_tuple: tuple[int, int, str]*. Each element must pass it's own type check upon initialization.dic

-dict takes in two types, for key and value. *dict[keyType, Valuetype]*. Ex. *dict[str, int]*

Union
-Allows a variable to be one of several types

-Syntax: *my_func(var_name: typeA | typeB):***Default Values & Possibly None**
-Can give function args default values, such as possibly "no value":-Syntax: *my_func(legal_name: str | None = None)*

Classes as Types
-Type can also be a class, in which case args must be instances of that class

Pydantic Models
-Pydantic is a Py data validation library. It allows you to declare complex "shapes" of data classes. All class attributes are typed vars.

-When you create an instance of the class with some values, Pydantic verifies the values match the required attribute types, and converts them to the appropriate type, if possible, then gives you an object with all that data.

-In order to make your class a Pydantic model, it must inherit from the Pydantic *BaseModel* class. Ex. *class MyClass(BaseModel)*

-Ex.

```
from datetime import datetime
from pydantic import BaseModel

classs User(BaseModel):
    id: int        name: str = "John Doe"
```

```
    signup_ts: datetime | None = None
    friends: list[int] = []

external_data = {
    "id": "123",
    "signup_ts": "2017-06-01 12:22",
     "friends": [1, "2", b"3"],
}

user = User(**external_data)
print(user)
# User id=123 name='John Doe' signup_ts=datetime.datetime(2017, 6, 1, 12, 22) friends=[1, 2, 3]
print(user.id)    # 123
```

Type Hints with Metadata Annotations
-Py 3.9+ includes *Annotated*, which allows you to include metadata for typehints, where this can provide additional info on a parameters

-Syntax:

```
from typing import Annotated

def say_hello(name: Annotated[str, "this is just metadata"]) -> str:          # type of name is str, plus str has metadata as provided by Annotated
```

Type Hints in FastAPI
-Due to FastAPI's use of type hinting and Pydantic, with FastAPI, you get editor auto-complete, type checks, and data conversion, data validation, type documentation in Swagger/ReDoc Docs, and FastAPI can define requirements from query params, bodies, etc.

00. Concurrency and async / await

Overview
-Py version of async code via *coroutines* is enabled by *async* and *await* usage.

Asynchronous Code
-Code that is safe to execute at the same time, even if one *async* part of code requires data from another *async* part of code. Python has a single execution thread. Thus need to use *async* and *await* to enable concurrency.

-In Python, even if one *async* piece of code needs data from the other *async* code, that is okay, due to *await* (concurrency).

-Async example: Computer runs long *async* script that needs external data mid way through. Script starts, and another *async* script starts generating external data. At same time, main script keeps running. When main scripts hits point where it needs external data, if external data still not ready, it will *await* for it, then continue running when it becomes available.

-Common reasons for *async*:

- data from client is being sent through network
- data from application is being sent to client through network
- contents of a disk file being parsed
- file being written to disk

- remote API operation
- database operations
- etc. latency causers

-Synchronous code must finish step to step, in an always repeating single order, where async allows concurrent code to execute.

-Parallelism, like concurrency, also says, "different things, happening, more or less, at the same time," but in a different manner.

Concurrent Burgers
-An example of concurrency:

You go to burger store and order two burgers from a single cashier. The cashier yells back to the cook, "two burgers!," and the cook takes note of the order, while continuing to work on other existing orders. While the cook keeps cooking, you pay, then the cashier gives you change. The burgers are not are not done yet, so you go sit at a table and chat with your friend. You listen for the cook to announce your order is ready, then go pick it up when it is. You then eat the completed burgers.

Parallel Burgers
-An example of parallelism:

You go a burger store and two burgers from a long line of many cashiers. Each cashier is also a cook, and after taking an order and payment, goes back to cook the order.

The burgers are not done yet, but you also have to wait at the counter for them to finish, as your cook does not have order numbers, as he only ever has one order at a time. The cook finishes cooking your food, brings it to you, then you go eat. You do not have time to chat with your friend during cooking, due to waiting at the counter at this time.

Parallelism vs Concurrency
-Parallelism occurs when multiple threads are available, all running across multiple processors (or distinct threads in a multi-core processor).

-Concurrency occurs when tasks can run together, as long as they stop and wait for data (or whatever) when they need it (or need it done) but some other task has not yet produced it.

-Benefit of concurrency over parallelism is that even though many of the same threads can execute together, you can't execute other related tasks, in the meantime. If a lot of waiting, concurrency thus makes sense, so can execute other tasks, while waiting.

-As web applications involve a lot of waiting, concurrency makes sense for many web applications.

-Server waiting on user to send requests. Users waiting on responses to come back. Request-response, is full of waiting. Even JS on the front-end is event-loop driven.

-On flip side, if no waiting required, just a lot of work to be done on different, unrelated tasks, then parallelism makes sense. Ex. eight rooms in a house each being cleaned by their own house cleaner, each working working together, in parallel.

-Parallel problems are, "CPU bound," as the more CPU threads you can run, the more parallel tasks can execute.

-Common reasons for parallelism:

- audio and image processing
- machine learning
- deep learning

FastAPI Concurrency + Parallelism
-NodeJS is also async, as is Go. Starlette (part of FastAPI basis) offers both parallelism and concurrency, thus making FastAPI faster than most NodeJS frameworks, and on par with Go.

-Concurrency comes from FastAPI use of *async* and *await*, which works via an event loop, just like in JS

-FastAPI also allows parallelism, via allowing multiple *Worker* processes to start on application launch, which then allows replication of processes to take advantage of multiple cores, to be able to handle more requests.

async* and *await
-Syntax: *async def my_function(arg: type): ...*
await my_function("my_arg")

-Nice to work with, as this allows the code to still read like it is running sequentially.

-In FastAPI, *async* allows execution on one request to *await*, while FastAPI takes care of another request

-All *async* functions must be called with *await*

-Ex. Async FastAPI Route:

```
@app.get('/burgers')
async def read_burgers():
    burgers = await get_burgers(2)
    return burgers
```

-As any function which contains *await* logic is waiting on an *async* function itself, that function must also be *async*.

-Ex.

```
async def get_data():
    ...


async def get_double_data():
    return [await get_data(), await get_data()]
```

FastAPI Async Basis
-Starlette, which is basis for FastAPI, is based on AnyIO, which makes FastAPI compatible with both python's standard *asyncio* library and Trio (a Py friendly library for async concurrency and I/O)

-You can directly use AnyIO for advanced concurrency use cases, both within FastAPI apps, and simply as it's own library, in non-FastAPI apps

-Also is Asyncer library, which improves type annotations, gets better auto-completions, provides better inline error handling, etc., compared using AnyIO alone. Useful for if need to combine async with sync code.

-All FastAPI standard *def sync* functions are run in the single standard Py external threadpool, with returns

without awaiting

-A component can own both async and sync functions

Coroutines
-The return of an async function. Similar to a JS promise. Something like a function, that Py can start at some point, and will end at some point, but that might pause, too, whenever an *await* occurs.

00. Environment Variables

-AKA Env var. A variable that lives outside of the back-end language (ex. Py), in the OS, and can be read by the back-ends

-Useful for holding application settings and config

-Can create env var by calling *export MY_NAME="Wade Wilson"*, after which can *echo "Hello $MY_NAME"*. Can also create an *.env* file, to store env var to use within application.

-Get OS env var with:

```
import os
os.getenv("NAME_NAME", "default_value")     #default for if MY_NAME var not settings
```

-As a env var are environment related, and stored out of program, do not need to commit to git

-Can create env var for program that only exists during program execution with: *MY_NAME="Wade Wilson" python main.py*

Types and Validation
-Env var can only be created as string, and thus are imported into Py app as string, also.***PATH* Env Var**
-PATH env var - a special env var used by the OS to find programs to run, such as commands which one might execute via the terminal

-On Linux and macOS, this is */usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin*, where each *:* is a delim for a diff directory to read from

-Should add *Python* to your *PATH* variable, so you can run it as a command, to start a program, etc.

00. Virtual Environments

-Virtual environments create virtual execution environments for programs, where each stores and manages their own dependencies. Any directory can be a virtual environment and all files within it are part of that venv.

-Create a virtual environment:

1. Make a directory and change to it
2. Run *python -m venv .venv* # runs py module *venv* and creates a venv in subdir *.venv*
   # can name *.venv* dir anything, such *.venvvvvvv*

-Activate virtual environment: *source .venv/bin/activate*

-Check v env is active: *which python*

-Deactivate v env: *deactivate*

-After create new *venv*, then active it, should upgrade *pip* via: *python -m pip install --upgrade pip*

-Make sure to add *.venv* to *.gitignore*, so does not commit

-Once in *venv*, can install requirements directly, via *pip install "fastapi[standard]"* or have a *requirements.txt* file that holds requirements to install via *pip install -r requirements.txt* :

```
// requirements.txt
fastapi[standard]==0.115.6
pydantic==2.10.4
```

-Once in v env, any commands run inside are run in v env, so if run *fastapi dev main.py* in venv, will execute a *fastapi* instance in that venv.

-Make sure to config your IDEf to use your project's venv, if it does not automatically, to ensure you get auto-completion from dependencies, ability to step into dependency code, etc..

-If do not use venv, *pip* installs are installed to global *venv*, which wastes OS space and does not properly encapsulate dependencies on a project-by-project basis. What happens, also, if two programs require the same dependency, but different versions? Also makes difficult to manage dependencies, when all installed in global env.

-When activate *venv*, as part of setting that v env active, *./venv/bin* is added as part of v env *PATH*, putting the *venv* directory at the start of *PATH*, so it is the first directory commands try to execute from. Ex.

/home/user/code/awesome-project/.venv/bin:/usr/bin:/bin:/usr/sbin:/sbin

-Plenty of other additional ways to manage projects, also, like uv library, which is a tool to manage the entire project, package dependencies, virtual envs, etc.. or Docker Compose.

01. FastAPI First Steps

Check it Out

-Start by creating a sample FastAPI project directory, then creating a python venv within it, then activating venv, and installing FastApi from within venv via: *pip install "fastapi[standard]"*

-The most basic FastAPI application contains an instance of the app and a single route within it:

```
#main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
```

return {"message": "Hello World"}

-Once you have a project with some basic FastAPI code, you can start the FastAPI dev server via command: *fastapi dev main.py* , which spins up a live server and gives you a URL to the server, and the auto-created user Swagger/ReDoc docs for the API.

-Local server at: http://127.0.0.1:8000
-Docs at: http://127.0.0.1:8000/docs

OpenAPI
-Schema - a definition of the layout for something
-API Schema - the paths of the API, the possible params an API path will take, etc.. In FastAPI, the schema is based on OpenAPI specifications.-Data Schema - the shape of the data, such as JSON content. FastAPI uses JSON schema, where attributes are also typed. API schema uses JSON schema as part of definition.

-API Schema is stored in *openapi.json* file created and updated by FastAPI, which can be viewed locally at: http://127.0.0.1:8000/openapi.json

-The OpenAPI schema powers the interactive docs. Many alts for interactive docs exist, and all can be integrated easily into FastAPI app by using OpenAPI schema as input.

-OpenAPI schema also useful for auto-generation of code, for clients that communicate with FastAPI, such as front-end.

Starter Details
-Based on the code seen in "Check it Out" section above

- *@app.get("/")* creates a path decorator for the root path. A decorator in FastAPI is the pattern in which you place behavior inside a wrapper for various HTTP operations, to give logic to those operations, for a specific path. FastAPI calls these path operation decorators.

-A path is a specific route in your API. For example, in *https://example.com/items/foo*, the path is */items/foo*. A path can also be called a "route" or "endpoint." Paths are the main way to separate concerns and resources in a REST API.

-Each path in FastAPI is based on a HTTP operation, including *POST, GET, PUT, DELETE, PATCH, HEAD, OPTIONS,* and *TRACE.* Each path can take in one or more of these operations, each with their own behavior.

-Ex. of path operation decorators: *@app.post(...), @app.put(...), @app.delete(...)*

-There is no enforcement for how you use each HTTP operation, but they should generally follow expectations for use. For *GraphQL* data, generally perform all actions via *POST*.

-Path operation decorators sit directly above the function def, as seen in previous example.

-Functions for path operation decorators can be *async* function, or just regular synchronous, depending on need.

-API paths can return anything, from *list* to *dict*, to *str*, to *int*, to more complex Pydantic models, to ORMs. For example, if you return *1*, raw data return will simply be *1*.

Basic FastAPI Recap
1. Import FastAPI
2. Create an *app* instance

3. Write a path operator decorator using a decorator like *@app.get("/mypath")*
4. Define a path operation function underneath the function (sync or *async)*
5. Run the dev server with *fastapi dev entry.py*

02. Path Params

Path Params
-Path params are values passed into the path when it is called. The path param then is also available inside the wrapped path function. Ex.

```
@app.get("/items/{item_id}")
async def read_item(item_id):
        return {"item_id": item_id}
```

-Note the *{}* in the above param. This is where the path param goes. Same syntax as typical python f-string, but without the *f*.

-Path params are usually used to identify a resource or set of resources

Typed Path Params
-Path params can also be given a type, via typical typing of the param inside function signature. Ex. *async def read_item(item_id: int):*-Typing params is pretty much always a good idea, due to better error checks, easier test writing, IDE auto-completion, etc.

-Types are also maintained in returns of FastAPI. Hence, in the above example, the return of calling with *item_id*=3 is *{ "item_id": 3 },* where 3 is an *int*, not a *str*.

-If you try and pass in the wrong type, FastAPI will auto-generate an error object for the return, with a descriptive error message, such as *"msg":"Input should be a valid integer, unable to parse string as an integer"*, and also denoting the error type, path, and input given. Very useful for debugging.

-Interactive FastAPI docs will also show required param types, and display UI errors, if wrong type passed.

-Pydantic provides the data validation

Path Declaration Order
-When you define multiple of the same path operation decorator HTTP types in the same file, order matters, as, if a call could match multiple paths, whichever path is defined first is hit.

-Example:

```
@app.get("/users/me")
async def read_user_me():
   return {"user_id": "the current user"}


@app.get("/users/{user_id}")
async def read_user(user_id: str):
   return {"user_id": user_id}    # if call above with mydomain.com/users/me, FastAPI could see this as
```

to call the */me* path, but could also see *me* as a *user_id*. Hence, whichever path was defined first would be called.

-As such, you should also not redefine paths using the same path location (ex. *@app.get("/users")* and *@app.get("/users")*), as the first one would always be hit, and the second one just never used pointless code.

Predefined Path Param Values
-If a path param should only accept certain values, you can define these using an *Enum*, via class which inherits from Py *Enum*.

-Ex.
```
class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep Learning FTW!"}

    if model_name.value == "lenet":
        return {"model_name": model_name, "message": "LeCNN all the images"}

    return {"model_name": model_name, "message": "Have some residuals"}        # example return
{  "model_name": "alexnet",  "message": "Deep Learning FTW!" }
}
```

-Note in above example, that path param has same type as created model value. As such, if pass in an invalid Enum name, where name is not included in Enum (ex. *"rogernet"*), FastAPI will return a type error message for it's response.

-When give a param a predefined set of values, API Docs will also generate a *select* menu for this param, only allowing these types in interactive UI.

-Note, in above example, that you can get the value of the Enum name passed in by calling *.value* on it.

-Can also return Enum names (*model_name* in the above example), which are then type cast to string.

Paths as Path Params
-What if you want to make a filepath itself a path param, such as */files/{file_path}* taking in *home/johndoe/myfile.txt* ? Rejected! This creates hard to test and define scenarios and OpenAPI does not allow this.-If you do wish to do a path value as a path param, use an internal tool from Starlette to add support for this.

Path Params Recap
-Thanks to FastAPI path param typing, you get:

- Editor support for error checks, auto-completion, etc.
- Data "parsing"
- Data validation
- API annotation and auto docs

03. Query Params

Query Params
-Query params allow you to make more detailed requests than path params alone. All other function params but

the first param (path param) defined within a wrapped path function sig are query params.

-Query params are usually used to filter or sort a resources or set of resources

-Example:

```
FAKE_DB_ITEMS = [{"item_name": "Foo"}, {"item_name": "Bar"}, {"item_name": "Baz"}]

@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]

# example call mydomain.com/items/?skip=0&limit=10 # returns {"skip": 10 }
```

-Query params in a HTTP address are defined after a *?* char, which follows the path, and which each param is separated by an & char, with values assigned to names via = char.

-Ex. *mypath/?skip=0&limit=10 #skip, with value 0, is first query param. limit, with val 10, is second.*

-All query params, as part of the URL, are strings, but when they are declared types within the path function sig, they are auto cast (and validated) into the proper type.

-The same editor support, data "parsing", data validation, and auto docs features available to path params via FastAPI also apply to query params

Default & Optional
-As query params are not a fixed part of the path, they are often optional, and thus can have default values for if no value provided. Typical Py syntax of *my_var: type = default_val*, for these.

-Params can also be optional, in which no default value is provided, and if value not passed in for param, value is simply *None*

-Optional param syntax varies depending on Py version:

- Py < 3.10: *from typing import Optional* *(my_var: Optional[type] = None)*
- Py >= 3.10:
  - *(my_var: type | None = None)*

Type Conversion-*bool* query params can be converted from numerous values. 1/0, True/False, true/false, on/off, and yes/no will all type cast to a proper True False *bool* value.**Required Params**
-Any param without a default value will be required. Not including it in the request will result in an error object being returned.

-Required and default params can be mixed, as per standard Py.

Enums
-In the same way Enums can be used to define "allowed value" params for path params, so can they be used for query params, also.

04. Request Body

Request Bodies
-Request query params can be used for simple data retrieving or filtering (ex. *http://example.com/api/data?*

id=123). All data sent via query params is also visible through the URL itself.

-Request body data is not visible via URL and is often used for sending larger or more complex amount of info, often in JSON, XML, etc. form.

-Ex. may use path param to get specific object, query param to filter a list of objects, and request body to POST an object

-Request bodies also key for POST, PUT, PATCH, etc.

-Data models in FastAPI inherit from Pydantic *BaseModel* class
-Example:

```
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

-After create Pydantic *BaseModel* inheriting class, can pass it in as a data model to FastAPI endpoint function, using model class name as type.

Request Body Docs
-FastAPI performs the same type conversion, data validation, type checking, JSON OpenAPI schema generation, auto doc generation, etc. on request bodies as it does path and query params.

Editor Support
-Also get type hints, error checking, and auto-completion in editor from declaring data classes. Thanks again to Pydantic for this.

-If using PyCharm for IDE, check out Pydantic PyCharm Plugin for even better Pydantic model support.

Using the Models
-Once a request body model has been passed into endpoint function, can access all props in it as you normally would (ex. *product.price*), then use to update DB data, etc.

Mixing Params
-Can take in path params, query params, and request body all at same time. To differentiate which is which:

- If endpoint function param also declared in path (ex. "/items/{item_id}"}, then function param is path
- If endpoint function param is primitive, then query param
- If endpoint function param is Pydantic model, then is request body

-Ex.

```
# item_id == path param, item == request body, q = query param

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, q: str | None = None):
    result = {"item_id": item_id, **item.dict()}

    if q:
```

result.update({"q": q})

return result

-Can also use *Body* parameters, instead of Pydantic models for request bodies. More on this later

Putting it all Together
-/this/is/my/path/{entity_id}/?query=param&another=12

-Path - */this/is/my/path/* - the API endpoint you are hitting-Path Param - *{entity_id}* - typically a unique identity(ies) identifier(s)
-Query Params - *?query=param&another=12* - typically used to sort and filter entities-Request Body - Not seen in URL. Part of HTTP request itself. Typically larger data for C & U.

-User makes a request to a specific path. The path could be general, such as just getting all items, or it could be for a limited number of item(s). When getting specific item(s), path param is used to get specific entity(ies). Query params can this pass in additional vars to help sort, limit, or filter results. Sensitive data and large data, like in POST and PUT/PATCH requests, is sent via the request body.

-FASTAPI paths are defined by function wrappers which also define actions for the path based on HTTP verbs. Path params are defined as part of the wrapper path. The wrapped functions knows which of it's signature params is the path param as that param also defines the path param identifier in the path (ex. *{entity_id}*). If an param is a primitive, it reads it in as a query param. If a param is an Pydantic model, it sees this param as a request body.

-Example: *@app.patch("/items/{item_id}")*
async def set_offer_if_item_expensive(item_id: int, item: Item, expensive_price: float) -> Item:
 # item_id arg == {item_id} part of path --> path param
 # item is type Item == Pydantic model --> query param # expensive_price == primitive --> query param

05. Query Params & String Validators

***Query* Validation**-Able to perform additional validation on query params easily in FastAPI. Done via a combo of *Annotated Python module* and FastAPI *Query*: *from typing import Annotated*
from fastapi import Query

@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
 # q can be str or None AND must be 50 chars or less
...

-*Annotated* - allows you to include metadata for typehints, where this can provide additional info on a parameters. *It both sets the type of the param and gives additional data for param, like setting validation with Query()* above. *q: str | None = None q: Annotated[str | None] = None # both mean the same thing-Query* - when pass a *Query* to the *Annotated* object it enforces this query on the path function param, hence having thus both type checking and more complex validation.-If request fails query validation, FastAPI will raise a clear error stating invalid client data. Param will also show the query validation in OpenAPI (Swagger, etc.) docs.

-Additional Query validations:

- *min_length=[int]*
- *pattern="^fixedquery$"*

Required & Default Values
-Can of course use default val besides *None* too: (*q: Annotated[str | None, Query(...)] = "default")* . And if don't want default value, just don't give it one in sig: (*q: Annotated[str, Query(...)])* .

-Can force client to send a default value, even if *None* by setting *None* as an optional type, but not giving arg default value via = *None* also. This forces client to send value w/ request even if val is *None*.

Query Param Lists
-If you define a query param as a list you can pass in multiple values for the param throughout a request:

```
@app.get("/items/")    async def read_items(q: Annotated[list[str] | None, Query()] = []):
    query_items = {"q": q}
    return query_items    # http://localhost:8000/items/?q=foo&q=bar        <-- q: ["foo", "bar"]
```

-If want specific size for list, good time to use *min_length* and *max_length Query* validations**General Param Metadata**
-*Query* can take in other args as well, such as:

- *title="Query title"*
- *description="Description of query string and some validation or whatever"*
- *alias="another-name-that-query-param-can-be-requested-as"*

-Ex. *... Query(title="Query Max", description="Ensure max query size not passed", max_length=75)]*

Depreciating Params-Not something you should do very often, but if do, and notify clients well in advance, via *Query(deprecated=True)*, which will then update docs to show param as deprecated

Hiding Params in OpenAPI
-Can also hide that a param exists for API in OpenAPI docs via: *Query(include_in_schema=False)*

Custom Validation
-Pydantic also provides a collection of custom validators, such as *AfterValidator*. This can also be passed into *Annotated,* instead of *Query,* to perform custom call-back function query param validation.

```
from pydantic import AfterValidator    …    data = { ... }
    def some_validation_function(validate_me: type):
    .…    @app.get("/items/")        async def read_items(id: Annotated[str | None,
AfterValidator(some_validation_function)] = None):          ....
```

06. Path Parameters and Numeric Validations

Path Validation-While *Query* FastAPI module allows you to perform validation on query params, FastAPI's *Path* module allows path validation.

-Ex.

```
from fastapi import Query
 ...
```

@app.get("/items/{item_id}") async def read_items(item_id: Annotated[int, Path("title="The ID of the item to get)]): ....

-*Path* works just like *Query* and takes in the same possible args (*min_length, max_length, title,* etc.) as *Query.*

Number Validation
-Can pass in args:

- *gt=1* – greater than 1
- *ge=0.5* – greater than or equal to 0.5
- *lt=1.2* – less than 1.2
- *le=100* – less than or equal to 100

-ex. *Annotated[int, Path(ge=0, le=100)]*

-Note that the above number validators can be also used to validate query params as args to *Query*

07. Query Param Models

Query Param Models
-Query param models - allow you to define your query params via a model

-ex.

class FilterParams(BaseModel):
limit: int = Field(limit=100, gt=0, le=100)
offset: int = Field(0, ge=0)
order_by: Literal["created_at", "updated_at"] = "created_at"
tags: list[str] = []

-When set param type as a Pydantic model, then attach a *Query* to it with *Annotated*, FastAPI defines expected query params by model-This should be prefered to defining params inline, as it allows proper separation and DDD layers, plus makes endpoint definitions less verbose and allows re-usability

-*Field* another Pydantic data model base class (like *BaseModel)* that takes in a variety of possible args such as *default, alias, strict, repr,* and constraints like the

-Query params defined by model props shows in OpenAPI docs just like inline defined params

-Can also forbid params from being submitted as part of query via param model prop of:
model_config = {"extra": "forbid"} # error raised if client submits params besides model props
-Note that cannot use Pydantic models for path params, as path params are primitive and thus do not need models.

08. Body – Multiple Params

Multiple Params
-FastAPI knows a Path type param is a path param, a Query type param is a query param, and a BaseModel param is a request body.

-Query and request body params can both be optional by giving a default value of None, but path param is required.

-Endpoint can take in multiple body params, too: *@app.put("/items/{item_id}")*
async def update_item(item_id: int, item: Item, user: User):
...
-All separate body objects passed into the endpoint are combined into a single FastAPI JSON object, where each request body passed in lives in this:

{
"item": { "item_prop": "value", ... },
"user": {
"user_prop": "value", ... } }

Body
-Query params can have extra data for them defined with FastAPI's *Query* module. Path params with *Path* module. Request body params with *Body* module. Ex.

from fastapi import Body

@app.put("/some_route")
async def update_item(request_body: Annotated[int, Body()] ...

Primitive Value Request Bodies
-If want to pass in a primitive (int, float, str, etc.) instead of an object to endpoint also can do this:

@app.put("/some_path")
async def update_item(item: Item, user: User, importance: Annotated[int, Body()]):
...

-FastAPI thus sees this as part of request body, as just another request body param, instead of seeing it as primitive and thus thinking it is a path param.

Embedded Body Param
-Typically when have single request body param passed into endpoint (ex. *Item*) the body param is expected to look like: *{ "item_prop": "value" .... }*

-But can instead pass in prop inside body, with name for prop referencing it's props inside an object in outermost body layer via *Body(embeded=True)*:

@app.put("some_route")
async def do_something(my_model: Annotated[MyModel, Body(embed=True)]):

{

"my_model": {
"item_prop": "value" ....
}
}

09. Body – Fields

Field Metadata
-In the same way can declare extra data on query, path, and request body params via *Query*, *Path*, and *Body* modules, the same can be done on *Field*.

-With field, no need to wrap in *Annotated* though:
description: str | None = Field(default=None, title="The description of the item", max_length=300)

-Field can take in all the same params as *Query*, *Path*, etc. (*le*, *gt*, *min_length*, *description*, etc.)

10. Body – Nested Models

Collection Props
-Pydantic models can contain props which reference collections of elements, such as *list[str]* or *set[str]*, etc.

-ex.
class Item(BaseModel): tags: set[str] = set()
weights: dict[str, float]

Nested Models
-Models can contain models as props

-ex.
class ProductImage(BaseModel):
url: str
name: str
class Item(BaseModel):
image: Image | None = None

-Model within model props get the same editor support, data conversion, validation, documentation, etc. like any other model in FastAPI

-Models can be deeply nested too. A model contains a model which contains a model, etc..

Special Types
-Pydantic provides a variety of special types, many of which inherit from existing types. For example, *HttpUrl* which verifies that the prop is a valid URL and is documented as URL in OpenAPI schema.

-Syntax: *from pydantic import HttpUrl ....*
Image(BaseModel):
url: HttpUrl

11. Declare Request Example Data

Request Example in Model
-Can define example inside model definition by adding *model_config* prop, which contains a *json_schema_extra* object, which contains and *examples* list of example objects:

```
model_config = {
    "json_schema_extra": {
        "examples": [
            {
                "name": "Foo",
                "description": "A very nice Item",
                "price": 35.4,
                "tax": 3.2,
            }
        ]
    }
}
```

-This example(s) will then show as example(s) in API docs

-Defining model schema examples this way is nice as any endpoint which uses the model as it's schema has this example**Param Module Examples**
-*Field* can also provide an example for primitives via *examples* param:

```
description: str | None = Field(default=None, examples=["Large red apple from IN"])
```

-Can also do the same with *Query, Path, Body, Header, Cookie, Form,* and *File* modules. Ex.

```
@app.put("/items/{item_id}")
async def update_item(
    item: Annotated[
        Item,
        Body(
            examples=[
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ],
```

```
        ),
    ],
):
```

-Can include multiple examples (inside) *Param* inheriting objects and in definition inside model

OpenAPI Examples
-Can also include examples built for OpenAPI specifically (include more fields inside example like *summary, description,* and *value,* which contains actual prop example data:

```
    Body(
        openapi_examples={
            "normal": {
                "summary": "A normal example",
                "description": "A **normal** item works correctly.",
                "value": {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                },
            },
    )
```

-Not much benefit to this for most use cases versus just given standard FastAPI examples

12. Extra Data Types

Additional Data Types
-*UUID* – Universally Unique Identifier common for ids in DB repr as *a* str in requests/responses. Import from *uuid*.

-*datetime.datetime* – py date and time

-*datetime.date* – py date

-*datetime.time* – py time

-*datetime.timedetla* – py timedelta

-*frozenset* – same as set but immutable

-*bytes* – py bytes

-*Decimal* – standard py decimal, handled same as float

12. Cookie Params

-Can tell endpoint that data being sent into endpoint is a cookie header via:
@app.get("/items/")
async def read_items(ads_id: Annotated[str | None, Cookie()] = None):
...-This says, look for cookie *ads_id* being sent over as a cookie header on incoming request to endpoint. It will contain a string (or *None*) value.

-Setting a cookie will be covered later in a doc. This just covers how to handle receiving a cookie as part of request.

-Cookies show in docs

13. Header Params

-Define the same way as other *Param* inheriting classes like *Query* and *Cookie*:
@app.get("/items/")
async def read_items(user_agent: Annotated[str | None, Header()] = None):
....

-Headers are commonly named using hypens (ex. *user-agent*) but python requires underscore for word separation in var names (ex. *user_agent*). Header thus auto-converts any hypens in the header name sent to underscores.

-Can disable underscore conversion by passing in param *convert_underscores* as *False* to *Header:*
Annotated[str | None, Header(convert_underscores=False)]

-Can also recieve duplicate headers, meaning same *Header* w/ multiple values, just like with *Query* by declaring the *Header* type as a collection:
Annotated[list[str] | None, Header()]

-Headers show in docs

13. Cookie Param Models

-If re-using a cookie across multiple endpoints, useful to declare a cookie model which inherits from *BaseModel* then declare the param as this type when annotating it as a *Cookie* param:

```
class Cookies(BaseModel):
    session_id: str
    facebook_tracker: str | None = None
    google_tracker: str | None = None

@app.get("/items/")
```

```
async def read_items(cookies: Annotated[Cookies, Cookie()]):
    ...
```